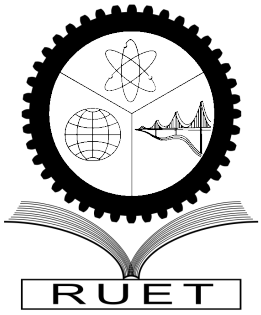
Rajshahi University of Engineering & Technology

Department of Electrical and Computer Engineering



MarketNest

Real-World Database System Design and Implementation

Multi-Vendor E-Commerce Database Management System

ECE 2216 – Database Systems Project

Submitted by: Mustakim Al Siyam
Student ID: 2310047
Registration No.: 1100
Submitted to: Md. Rakibul Islam
Lecturer
Department of Electronics & Telecommunication Engineering
Submission date: 10 November 2026

Contents

1 Introduction **1**

1.1 Problem Statement . . . 1

1.2 Motivation and System Selection . . . 1

1.3 Objectives . . . 1

1.4 Scope and System Boundary . . . 2

1.5 System Users . . . 2

1.6 Functional Requirements . . . 2

1.7 Non-Functional Requirements . . . 3

1.8 Business Rules . . . 3

2 Database Implementation **5**

2.1 Design Method and ER Model . . . 5

2.2 Relationship and Cardinality Summary . . . 8

Implemented Relational Schema . . . 9

2.3 Integrity Constraints . . . 11

2.4 Functional Dependencies and Normalization . . . 12

2.4.1 Unnormalized Form and Candidate Key . . . 12

2.4.2 First Normal Form . . . 13

2.4.3 Second Normal Form . . . 13

2.4.4 Third Normal Form . . . 13

2.4.5 BCNF Assessment . . . 14

2.5 Sample Data Coverage . . . 14

2.6 Database Setup and File Organization . . . 14

3 SQL Queries **16**

3.1 Query Coverage . . . 16

3.2 Business Questions Answered . . . 16

4 Advanced Database Features **18**

4.1 Views . . . 18

4.2 Stored Procedures . . . 18

4.3 Triggers . . . 19

4.4 Transaction Processing and Error Handling . . . 19

5 Reports and Application **20**

5.1 Application Architecture . . . 20

5.2 Implemented Operations . . . 20

5.3 Reports and Data Visualizations . . . 20

5.4 Dashboard Screenshots . . . 20

5.5 System Demonstration . . . 22

6 Testing **23**

6.1 Test Strategy and Results . . . 23

6.2 Detailed Test Cases . . . 23

6.3 Test Coverage and Error Handling . . . 26

7 Conclusion **27**

7.1 Achievements . . . 27

7.2 Limitations . . . 27

7.3 Future Improvements . . . 27

References **28**

List of Figures

2.1 MarketNest conceptual ER diagram. The notation key explains entity types, weak entities, identifying relationships, attributes, keys, participation, and cardinalities. 6

2.2 MarketNest relational EER diagram produced from the implemented database design. 7

5.1 MarketNest overview with KPI cards, monthly sales, and order-status charts. . . . 21

5.2 Checkout preview showing coupon savings and the updated total before order placement. 21

List of Tables

2.2 Implemented relational schema, keys, and principal constraints 9
2.4 Sample row counts for the 18 relations 15

3.1 Primary classification of the 29 SQL queries 16
3.2 Purpose of each labeled SQL query 17

6.1 Observed test execution summary 23
6.2 Test case inputs, expected results, observed results, and status 23

Chapter 1

Introduction

1.1 Problem Statement

Small and medium-sized sellers may manage products, customers, orders, and delivery information through separate spreadsheets, messages, and marketplace tools. This fragmented approach can cause inconsistent stock quantities, outdated prices, duplicated records, unclear payment and delivery status, and difficulty calculating seller commissions and payouts. Customers may consequently receive unreliable information about availability, order progress, and returns.

MarketNest is a multi-vendor e-commerce marketplace in which independent sellers list products and customers purchase products through a common checkout process. The platform records orders, payment attempts, courier shipments, returns, reviews, and seller settlements in a centralized relational database. The database is intended to maintain consistency and support the main operational and reporting activities of the marketplace.

1.2 Motivation and System Selection

A multi-vendor marketplace is sufficiently complex for demonstrating relational database concepts. Its operations involve multiple related entity sets, one-to-many and many-to-many relationships, composite keys, hierarchical categories, historical order-line data, and financial records. In particular, the system must preserve the price paid at checkout even if a seller changes the current catalogue price later.

1.3 Objectives

1. Store customer, seller, product, order, payment, shipment, review, return, and payout information in structured relations.
2. Maintain variant-level prices and stock quantities and prevent negative stock.
3. Preserve historical purchase prices and the delivery address selected for each order.
4. Support cart, wishlist, checkout, payment recording, fulfilment, review, and return workflows.

5. Calculate seller commissions and payouts from delivered sales.
6. Enforce data integrity using keys, constraints, triggers, procedures, and transactions.
7. Provide operational searches, reports, and visualizations through a dashboard.

1.4 Scope and System Boundary

The implementation uses synthetic sample data and a locally configured MySQL database. It demonstrates core marketplace database operations. It does not connect to a real payment gateway, execute courier bookings, authenticate dashboard users, calculate live carrier charges, support multi-warehouse inventory, or provide a complete bank settlement and refund ledger. Payment records are stored locally for demonstration; no card details are stored.

1.5 System Users

User or actor	**Main responsibilities**
Customer	Searches products, manages addresses, cart and wishlist, places orders, views delivery progress, reviews delivered items, and requests returns.
Seller	Maintains store and product listings, creates variants, updates price and stock, inspects sales, and reviews commission and payout information.
Administrator	Oversees seller and catalogue records, categories, coupons, couriers, orders, returns, payouts, and marketplace reports.
Courier staff	Updates shipment and delivery status for parcels assigned to the courier company.
Payment service (conceptual external actor)	In a real deployment, returns payment-attempt outcomes and transaction references. In this project, outcomes are recorded locally rather than obtained from a live gateway.

1.6 Functional Requirements

1. Maintain customer and seller records and customer delivery addresses.
2. Maintain categories, seller-owned products, and product variants with unique SKU, price, and stock.
3. Search and filter products by keyword, category, seller, price, and rating.
4. Create and manage customer carts and wishlists.
5. Create orders from cart items, apply eligible coupons, calculate totals, and reserve stock.

6. Record payment attempts, payment method, amount, status, and optional transaction reference.
7. Support order confirmation, cancellation before shipment, and stock restoration on eligible cancellation.
8. Record courier, shipment, tracking number, and shipment status.
9. Permit reviews only for delivered order items.
10. Process return requests subject to the delivery date and purchased quantity.
11. Calculate seller gross sales, commission, and net payout for a date range.
12. Generate operational and analytical reports and visualizations.
13. Support create, search and filter, update, guarded delete, and reporting operations in the dashboard.

1.7 Non-Functional Requirements

Quality	**Requirement**
Integrity	Keys and constraints prevent invalid references, duplicate identifiers, negative stock, invalid quantities, and invalid monetary values.
Consistency	Multi-step checkout and cancellation operations use transactions so partial changes are not committed.
Security	Credentials are configured locally; the example environment file contains placeholders. Production authentication and full role separation are outside the current dashboard scope.
Performance	Indexes support joins and common filters; views simplify recurring reporting queries.
Maintainability	SQL schema, seed data, database features, query set, application, and tests are separated into documented files.
Recoverability	Database backups should be taken before destructive maintenance or a clean re-seed.

1.8 Business Rules

1. Each order belongs to one customer and references an address belonging to that same customer.
2. A customer may maintain multiple addresses; an address is identified by the customer and its address number.
3. A customer has at most one cart in the implemented design.
4. A product belongs to one seller and one category; a seller and a category may each be associated with many products.

5. A product can have multiple variants. Each variant has its own unique SKU, price, and stock quantity.
6. Cart items and wishlist items are associative relations that prevent duplicate item pairs.
7. An order contains order lines; the same product variant cannot be repeated as a separate line in the same order.
8. Each order line stores the unit price paid at checkout, preserving historical price information.
9. Stock cannot become negative. A new order line is rejected if requested quantity exceeds available stock.
10. An eligible cancellation before shipment restores reserved stock.
11. Coupon validity, minimum spend, discount amount, and usage limit are checked when an order is created.
12. A review requires delivery and is limited to one review per order line; rating is from 1 to 5.
13. A return must be requested within seven days of delivery. Non-rejected return quantities cannot exceed the quantity purchased.
14. Seller commission is deducted from gross sales to calculate net payout. Payout date ranges for one seller cannot overlap.

Chapter 2

Database Implementation

2.1 Design Method and ER Model

The conceptual ER design contains 18 meaningful entities and associative relations. It records entity attributes, primary keys, relationship names, cardinalities, participation constraints, weak entities, and identifying relationships. Address, product variant, and order item are modeled with identifying keys; cart items and wishlist items resolve many-to-many relationships. The design meets the brief's requirement for a complete ER model with more than ten meaningful entities.

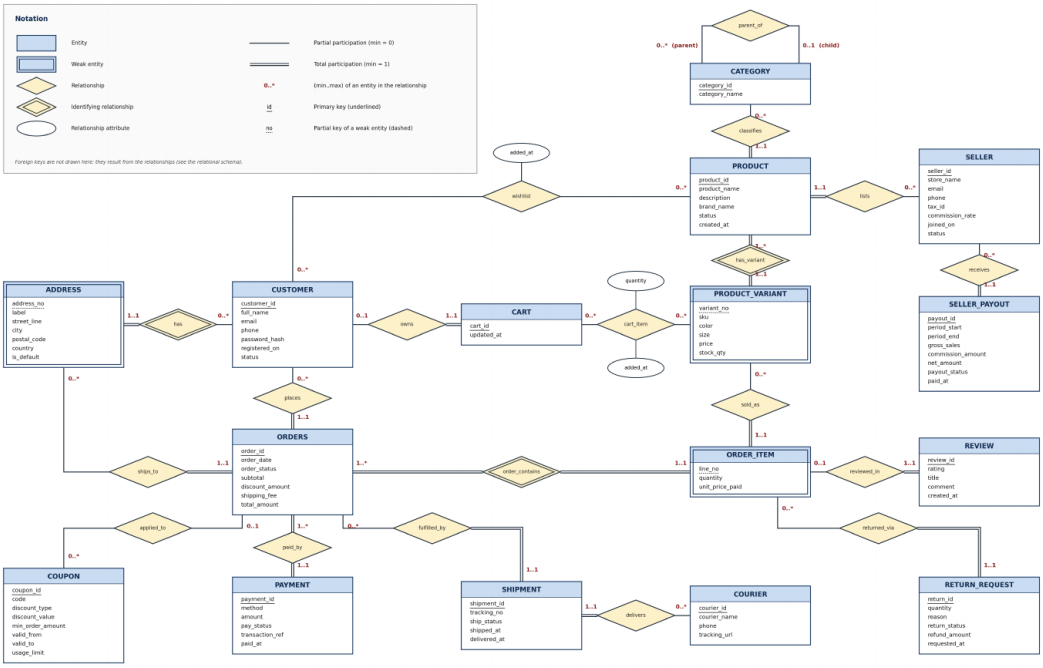


Figure 3.1: MarketNest Conceptual ER Diagram

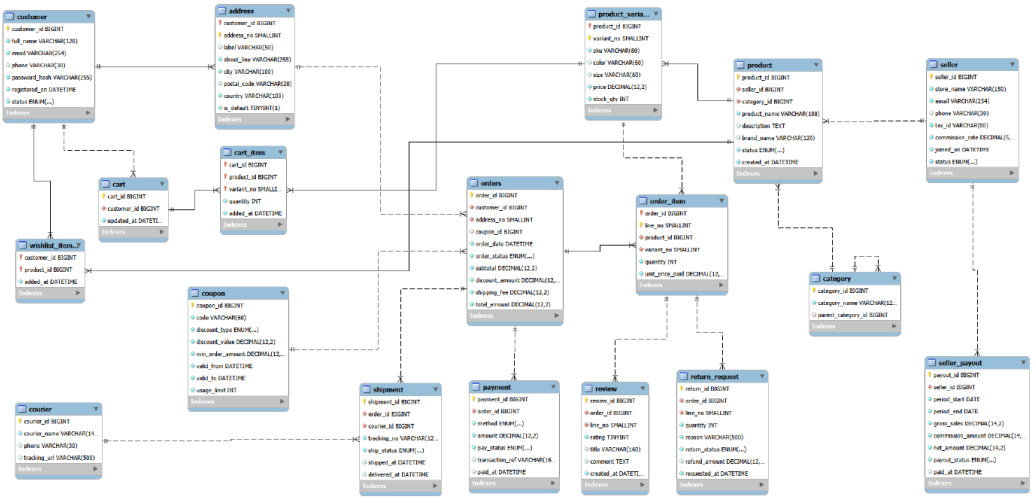


Figure 3.2: MarketNest Relational (EER) Diagram from MySQL Workbench

The relational EER diagram maps the conceptual model to the implemented MySQL tables. The detailed table definitions in `sql/01_schema.sql` are authoritative where a diagram label is shortened for readability.

2.2 Relationship and Cardinality Summary

Relationship	**Cardinality**	**Interpretation**
CUSTOMERS–ADDRESSES	1:N	A customer may save multiple addresses; each address belongs to one customer.
CUSTOMERS–CARTS	1:0..1	A customer has at most one cart in the implemented design.
CARTS – PRODUCT VARIANTS	M:N	Resolved by CART ITEMS, which records quantity.
CUSTOMERS–PRODUCTS	M:N	Resolved by WISHLIST_ITEMS.
SELLERS–PRODUCTS	1:N	Each product is owned by one seller.
CATEGORIES–PRODUCTS	1:N	Each product references one category; a category may contain many products.
CATEGORIES–CATEGORIES	1:N recursive	A category may have child categories.
PRODUCTS – PRODUCT VARIANTS	1:N	Each variant belongs to one product; a product may have multiple variants.
CUSTOMERS–ORDERS	1:N	Each order belongs to one customer.
ORDERS – ORDER ITEMS	1:N	Each order line belongs to one order.
ORDERS–PAYMENTS	1:N	An order may have multiple payment attempts.
ORDERS–SHIPMENTS	1:N	An order may be fulfilled in one or more parcels.
COURIERS–SHIPMENTS	1:N	Each shipment is assigned to one courier.
ORDER_ITEMS–REVIEWS	1:0..1	A delivered order line may have at most one review.
ORDER ITEMS – RETURN REQUESTS	1:N	One purchased line may have return requests subject to cumulative quantity rules.
SELLERS – SELLER PAYOUTS	1:N	A seller may have multiple payout records for non-overlapping periods.

Implemented relational schema. The table lists every attribute, including generated fields, and identifies primary keys, foreign keys, alternate keys, and principal constraints. Detailed data types, defaults, indexes, and named constraints are in the schema SQL file.

Table 2.2: Implemented relational schema, keys, and principal constraints

Table relation	Attributes, keys and constraints
CUSTOMERS	**Attributes:** `customer_id`, `full_name`, `email`, `phone`, `password_hash`, `registered_on`, `status` **Primary key:** `customer_id` **Foreign / alternate keys:** Alternate key: `email` **Constraints:** Required name, email, and password hash; status has defined values.
ADDRESSES	**Attributes:** `customer_id`, `address_no`, `label`, `street_line`, `city`, `postal_code`, `country`, `is_default`, `default_customer_id` **Primary key:** `customer_id`, `address_no` **Foreign / alternate keys:** `customer_id` references CUSTOMERS **Constraints:** Generated default-address key is unique; default flag is 0 or 1.
SELLERS	**Attributes:** `seller_id`, `store_name`, `email`, `phone`, `tax_id`, `commission_rate`, `joined_on`, `status` **Primary key:** `seller_id` **Foreign / alternate keys:** Alternate keys: `email`, `tax_id` **Constraints:** Commission rate is between 0 and 100; status has defined values.
CATEGORIES	**Attributes:** `category_id`, `category_name`, `parent_category_id` **Primary key:** `category_id` **Foreign / alternate keys:** `parent_category_id` references CATEGORIES; alternate key: `category_name` **Constraints:** Self-parenting is rejected by triggers.
PRODUCTS	**Attributes:** `product_id`, `seller_id`, `category_id`, `product_name`, `description`, `brand_name`, `status`, `created_at` **Primary key:** `product_id` **Foreign / alternate keys:** `seller_id` references SELLERS; `category_id` references CATEGORIES **Constraints:** Seller and category are required; product name/category indexes support search.
PRODUCT VARIANTS	**Attributes:** `product_id`, `variant_no`, `sku`, `color`, `size`, `price`, `stock_qty` **Primary key:** `product_id`, `variant_no` **Foreign / alternate keys:** `product_id` references PRODUCTS; alternate key: `sku` **Constraints:** Price and stock are non-negative; SKU is unique.
CARTS	**Attributes:** `cart_id`, `customer_id`, `updated_at` **Primary key:** `cart_id` **Foreign / alternate keys:** `customer_id` references CUSTOMERS; `customer_id` is unique **Constraints:** Enforces at most one cart per customer.

Table relation	Attributes, keys and constraints
CART ITEMS	**Attributes:** cart_id, product_id, variant_no, quantity, added_at **Primary key:** cart_id, product_id, variant_no **Foreign / alternate keys:** cart_id references CARTS; variant key references PRODUCT_VARIANTS **Constraints:** Quantity must be positive; duplicate cart/variant pairs are prevented by the key.
WISHLIST ITEMS	**Attributes:** customer_id, product_id, added_at **Primary key:** customer_id, product_id **Foreign / alternate keys:** customer_id references CUSTOMERS; product_id references PRODUCTS **Constraints:** Composite key prevents duplicate saved products.
COUPONS	**Attributes:** coupon_id, code, discount_type, discount_value, min_order_amount, valid_from, valid_to, usage_limit **Primary key:** coupon_id **Foreign / alternate keys:** Alternate key: code **Constraints:** Positive discount and usage limit; percent is at most 100; valid date range and minimum are checked.
ORDERS	**Attributes:** order_id, customer_id, address_no, coupon_id, order_date, order_status, subtotal, discount_amount, shipping_fee, total_amount **Primary key:** order_id **Foreign / alternate keys:** customer_id references CUSTOMERS; (customer_id, address_no) references ADDRESSES; optional coupon_id references COUPONS **Constraints:** Amounts are non-negative; discount cannot exceed subtotal; total equals subtotal minus discount plus shipping.
ORDER ITEMS	**Attributes:** order_id, line_no, product_id, variant_no, quantity, unit_price_paid **Primary key:** order_id, line_no **Foreign / alternate keys:** order_id references ORDERS; variant key references PRODUCT_VARIANTS; alternate key: (order_id, product_id, variant_no) **Constraints:** Quantity is positive; paid price is non-negative; variant may occur once per order.
PAYMENTS	**Attributes:** payment_id, order_id, method, amount, pay_status, transaction_ref, paid_at **Primary key:** payment_id **Foreign / alternate keys:** order_id references ORDERS; alternate key: non-null transaction_ref **Constraints:** Amount must be positive; payment method and status use defined values.
COURIERS	**Attributes:** courier_id, courier_name, phone, tracking_url **Primary key:** courier_id **Foreign / alternate keys:** Alternate key: courier_name **Constraints:** Courier name is required and unique.

Table relation	Attributes, keys and constraints
SHIPMENTS	**Attributes:** `shipment_id`, `order_id`, `courier_id`, `tracking_no`, `ship_status`, `shipped_at`, `delivered_at` **Primary key:** `shipment_id` **Foreign / alternate keys:** `order_id` references ORDERS; `courier_id` references COURIERS; alternate key: `tracking_no` **Constraints:** Tracking number is unique; delivery cannot precede shipping.
REVIEWS	**Attributes:** `review_id`, `order_id`, `line_no`, `rating`, `title`, `comment`, `created_at` **Primary key:** `review_id` **Foreign / alternate keys:** (`order_id`, `line_no`) references ORDER_ITEMS; this pair is unique **Constraints:** Rating is 1 to 5; trigger requires a delivered item.
RETURN REQUESTS	**Attributes:** `return_id`, `order_id`, `line_no`, `quantity`, `reason`, `return_status`, `refund_amount`, `requested_at` **Primary key:** `return_id` **Foreign / alternate keys:** (`order_id`, `line_no`) references `ORDER_ITEMS` **Constraints:** Triggers enforce delivery window, cumulative quantity, and valid status transitions.
SELLER PAYOUTS	**Attributes:** `payout_id`, `seller_id`, `period_start`, `period_end`, `gross_sales`, `commission_amount`, `net_amount`, `payout_status`, `paid_at` **Primary key:** `payout_id` **Foreign / alternate keys:** `seller_id` references SELLERS; alternate key: (`seller_id`, `period_start`, `period_end`) **Constraints:** Commission and net totals are checked; payout periods cannot overlap for a seller.

2.3 Integrity Constraints

The schema uses all constraint classes required by the brief. Primary keys identify every row; foreign keys enforce valid relationships; `NOT NULL` marks mandatory attributes; `UNIQUE` protects identifiers such as email, SKU, coupon code, courier name, and tracking number; `CHECK` restricts quantities, prices, ratings, commission rates, discounts, and totals; and defaults supply appropriate timestamps and statuses. Triggers and stored procedures enforce cross-row rules such as stock reservation and restoration, lifecycle transitions, delivery eligibility, return limits, coupon calculations, and payout-period overlap.

The order total follows total = subtotal − discount + shipping. A composite foreign key ensures that an order cannot use an address belonging to another customer. A generated unique key supports at most one default address per customer. Multi-row and concurrent operations use transactions or locking. Minimum participation rules, such as requiring an order to contain at least one order item, are validated by the checkout workflow.

2.4 Functional Dependencies and Normalization

A combined order-entry relation is selected to demonstrate normalization. It contains order-header data and a repeating group of purchased product variants. The relation is an instructional view of the marketplace data; the physical schema remains the authoritative implementation.

2.4.1 Unnormalized Form and Candidate Key

Listing 2.1: Illustrative unnormalized order form

```
ORDER_FORM_UNF(
  order_id, order_date, customer_id, customer_name, customer_email,
  address_no, ship_city, ship_postal_code, coupon_code,
  { sku, product_id, product_name, color, size, seller_id,
    seller_name, category_id, category_name, quantity, unit_price_paid }
)
```

The braces represent a repeating set of order lines. The candidate key after flattening the lines is (order_id, sku): a variant may appear on many orders, and an order may contain many variants; the implementation also prevents the same variant from being repeated as a separate line in one order.

The functional dependencies used in this example are:

ID	Functional dependency	Reason
FD1	order_id → order_date, customer_id, address_no, coupon_code	One order has one header.
FD2	customer_id → customer_name, customer_email	Customer ID identifies the customer.
FD3	(customer_id, address_no) → ship_city, ship_postal_code	Address number is scoped to a customer.
FD4	coupon_code → discount_type, discount_value	Coupon code identifies its rule.
FD5	sku → product_id, color, size	SKU identifies a sellable variant.
FD6	product_id → product_name, seller_id, category_id	Product ID identifies its listing.
FD7	seller_id → seller_name	Seller ID identifies a seller.
FD8	category_id → category_name	Category ID identifies a category.
FD9	(order_id, sku) → quantity, unit_price_paid	A variant occurs at most once per order.

The closure of (order_id, sku) determines the order header, customer and address facts, coupon facts, variant and product facts, seller and category names, quantity, and paid price. Neither component alone determines all attributes, so the key is minimal under these dependencies.

2.4.2 First Normal Form

The repeating group is removed by storing one product line per row. Every field contains one atomic value.

Listing 2.2: Order form in first normal form

```
ORDER_FORM_1NF(
 order_id, sku, order_date, customer_id, customer_name, customer_email,
 address_no, ship_city, ship_postal_code, coupon_code,
 discount_type, discount_value, product_id, product_name, color, size,
 seller_id, seller_name, category_id, category_name,
 quantity, unit_price_paid
)
-- Candidate key: (order_id, sku)
```

Partial dependencies remain: order-header facts depend only on `order_id`, while variant and product facts depend on `sku`.

2.4.3 Second Normal Form

A relation is in 2NF when it is in 1NF and every non-prime attribute depends on the whole candidate key. The partial dependencies are removed by separating order-header, variant, and order-line facts.

Listing 2.3: Decomposition into second normal form

```
ORDER_HEADER(
 order_id PK, order_date, customer_id, customer_name, customer_email,
 address_no, ship_city, ship_postal_code, coupon_code,
 discount_type, discount_value
)
VARIANT_INFO(
 sku PK, product_id, product_name, color, size,
 seller_id, seller_name, category_id, category_name
)
ORDER_LINE(order_id PK/FK, sku PK/FK, quantity, unit_price_paid)
```

The non-key attributes in `ORDER_LINE` depend on the complete composite key. Transitive dependencies remain in `ORDER_HEADER` and `VARIANT_INFO`.

2.4.4 Third Normal Form

The remaining transitive dependencies are removed by separating customer, address, coupon, product, seller, and category descriptions.

Listing 2.4: Illustrative third normal form decomposition

```
CUSTOMER(customer_id PK, customer_name, customer_email)
ADDRESS(customer_id PK/FK, address_no PK, ship_city, ship_postal_code)
COUPON(coupon_code PK, discount_type, discount_value)
ORDERS(order_id PK, order_date, customer_id FK, address_no,
       coupon_code FK, FK(customer_id,address_no) REFERENCES ADDRESS)
ORDER_LINE(order_id PK/FK, sku PK/FK, quantity, unit_price_paid)
PRODUCT_VARIANT(sku PK, product_id FK, color, size)
PRODUCT(product_id PK, product_name, seller_id FK, category_id FK)
SELLER(seller_id PK, seller_name)
CATEGORY(category_id PK, category_name)
```

This decomposition separates facts about each entity from order-line facts. The implemented physical schema uses `coupon_id` rather than coupon code in `orders`; it uses `(order_id,line_no)` as the order-line primary key and separately enforces uniqueness of the order/variant combination. The normalization example is a design analysis, not a replacement for the physical schema.

2.4.5 BCNF Assessment

A relation is in BCNF if every non-trivial functional dependency $X \rightarrow Y$ has determinant X as a superkey. For the decomposed relations above, the determinants are candidate keys under the stated dependency set: customer ID (and unique email) for CUSTOMER; $(customer_id, address_no)$ for ADDRESS; coupon code for COUPON; order ID for ORDERS; $(order_id, sku)$ for ORDER_LINE; SKU for PRODUCT_VARIANT; product ID for PRODUCT; seller ID for SELLER; and category ID for CATEGORY. Thus, the listed decomposed relations satisfy BCNF under these assumptions. This conclusion is limited to the illustrative relation and stated dependencies.

2.5 Sample Data Coverage

The deterministic synthetic dataset contains at least ten rows in every implemented relation, meeting the project brief's data-volume guideline. The counts below come from the included seed data and report summary. All sample accounts and payment references are synthetic. Customer and seller email addresses use the reserved `example.test` domain; no card details are included.

2.6 Database Setup and File Organization

The project uses MySQL 8.0.16 or later with InnoDB. MySQL Workbench can be used to run and inspect the scripts. Execute the SQL files in the following order from the project root:

1. `sql/01_schema.sql` creates/selects the database and the 18 tables.
2. `sql/02_seed_data.sql` inserts deterministic sample records. It is intended for a fresh database because it uses fixed IDs.
3. `sql/03_database_features.sql` installs views, stored procedures, and triggers.

Table 2.4: Sample row counts for the 18 relations

Relation	Rows	Relation	Rows
customers	10	addresses	15
sellers	10	categories	12
products	20	product_variants	30
carts	10	cart_items	20
wishlist_items	20	coupons	10
orders	15	order_items	20
payments	15	couriers	10
shipments	13	reviews	10
return_requests	10	seller_payouts	10

4. Review or execute `sql/04_queries.sql`. `sql/05_security_template.sql` is an optional least-privilege template and should be adapted by an administrator.

The feature script can be re-applied to replace its database objects. Back up data before a clean reset or destructive maintenance. Connection credentials are stored in a local `.env`; the project archive contains only `.env.example` with placeholders.

Chapter 3

SQL Queries

3.1 Query Coverage

The project contains 29 business-focused SQL queries in `sql/04_queries.sql`. Each query addresses a marketplace question. The table assigns each query one primary category so the counts sum to 29; three of the five aggregate queries also use `GROUP BY` with `HAVING`, satisfying that separate requirement.

Table 3.1: Primary classification of the 29 SQL queries

Category	Count	Queries	Concepts demonstrated
Basic queries	4	Q01–Q04	Selection, filtering, sorting, and range conditions.
JOIN queries	7	Q05–Q11	INNER JOIN, LEFT JOIN, and multi-table joins across marketplace relations.
Aggregate queries	5	Q12–Q16	COUNT, SUM, AVG, MIN/MAX, grouping; three use GROUP BY and HAVING.
Subqueries	4	Q17–Q20	Scalar, derived-table, and existence-based nested queries.
Arithmetic and logical	3	Q21–Q23	Computed revenue/commission and combined predicates.
Complex analytical	6	Q24–Q29	Common table expressions, rankings, LAG, CUME_DIST, and return rates.
Total	**29**	Q01–Q29	Meets the minimum query count and category requirements.

3.2 Business Questions Answered

Table 3.2: Purpose of each labeled SQL query

Query	Question answered
Q01	Which active catalogue variants have stock available?
Q02	Which orders were placed during a selected date range?
Q03	Which variants fall within a selected price range?
Q04	Which product variants have low stock?
Q05	What are a customer's order history and selected address?
Q06	Which seller and category are associated with each product and its stock?
Q07	What items were purchased and at what historical unit prices?
Q08	What are the shipment tracking details and courier contacts?
Q09	What payment attempts and totals are associated with each order?
Q10	Which purchased items have return requests and what are their states?
Q11	Which products have no reviews?
Q12	What are delivered sales by seller above a specified threshold?
Q13	Which categories contain multiple product listings?
Q14	What are the monthly order counts and average basket values?
Q15	Which product ratings meet a selected average-rating threshold?
Q16	How many return requests exist in each status?
Q17	Which products are priced above the average product price?
Q18	Which customers exceed the average delivered spend?
Q19	Which sellers have low-stock product variants?
Q20	Which products have delivered sales?
Q21	What are line revenues and the corresponding commission amounts?
Q22	Which orders have pending or failed payment attempts?
Q23	Which active listings have low stock?
Q24	Which products rank highest by delivered quantity?
Q25	How do sellers rank by monthly revenue?
Q26	How does monthly revenue change from the preceding month using LAG?
Q27	How do coupon usage and order values compare?
Q28	How does a customer's spend percentile rank using CUME_DIST?
Q29	What is the product return rate relative to delivered sales?

The executable SQL statements are maintained in the project bundle instead of being duplicated in full here. This keeps the report readable and leaves the SQL file as the authoritative runnable implementation.

Chapter 4

Advanced Database Features

4.1 Views

Three views provide reusable reporting datasets for the application and SQL analysis.

View	Purpose
`v_product_catalog`	Combines product, seller, category, variant, stock, and review summary for catalogue searching.
`v_order_summary`	Presents order totals together with successful payment sums and shipment counts.
`v_seller_monthly_sales`	Summarizes delivered sales, estimated commission, and net sales by seller and month.

4.2 Stored Procedures

Three stored procedures perform multi-step database operations.

Procedure	Operation
`sp_reserve_order_line`	Locks a variant, checks stock, inserts an order line with its paid price, and decreases stock within the caller's transaction.
`sp_cancel_order`	Performs an eligible cancellation for an unshipped order and coordinates payment-state changes with stock restoration.
`sp_generate_seller_payout`	Calculates delivered sales for a seller and inclusive date range, derives commission, inserts a payout, and returns its identifier.

4.3 Triggers

Thirteen triggers enforce cross-row rules and state transitions. Their names and roles are summarized below; the full definitions are in `sql/03_database_features.sql`.

Trigger	Responsibility
`bi_orders_validate_coupon`	Checks coupon validity, minimum spend, and discount amount on new orders.
`bi_order_item_check_stock`	Locks and checks a variant before a new order line is inserted.
`bu_orders_validate_transition`	Rejects invalid order lifecycle changes and cancellation after shipment.
`au_orders_restore_cancelled_stock`	Restores each line's reserved stock after eligible cancellation.
`bi_reviews_require_delivery`	Requires the related order item to have been delivered before review.
`bi_returns_validate_request`	Checks delivery, the seven-day request window, and cumulative return quantity.
`bu_returns_validate_quantity`	Preserves the purchased-quantity limit when a return request is updated.
`bu_returns_validate_transition`	Restricts return status changes to the allowed lifecycle.
`au_returns_restock_refunded`	Restores returned inventory when an approved return is refunded.
`bi_payouts_no_overlap`	Rejects overlapping payout periods when a payout is inserted.
`bu_payouts_no_overlap`	Rejects overlapping payout periods when a payout is updated.
`ai_categories_no_self_parent`	Prevents a new category from being its own parent.
`bu_categories_no_self_parent`	Prevents an update from making a category its own parent.

4.4 Transaction Processing and Error Handling

Checkout uses an application-managed transaction. The application records the order and its lines, reserves stock through the stored procedure, records the payment outcome, and clears the cart before commit. If a line has insufficient stock or another required step fails, the application rolls back the whole checkout so partial changes are not retained. Cancellation uses row locking and transaction handling so the order-state change, payment-state change, and stock restoration occur together. Product creation and its first variant are also committed together.

The database returns constraint or trigger errors for invalid input, duplicate values, missing references, and invalid state changes. The application reports the error to the operator and rolls back an active transaction. The interface validates required fields and numeric ranges before submitting changes; foreign keys and database constraints remain the final integrity boundary. Credentials are stored outside the source in a local environment file.

Chapter 5

Reports and Application

5.1 Application Architecture

The administrator operations dashboard is implemented in Python with Streamlit and connects to MySQL through the project database module. Connection settings are loaded from a local `.env` file. The submitted `.env.example` contains placeholders only. The real local `.env` file is excluded from the submission archive.

5.2 Implemented Operations

The dashboard demonstrates create, search and filter, update, guarded delete, checkout, inventory, fulfilment, returns, payments, and reporting operations. It supports seller and product creation, category and SKU management, color and size variants, customer and cart maintenance, coupon selection with a live discount and total preview, manual inventory adjustments, order status updates, return/refund handling, reviews, seller payouts, and chart-based reports. A manual Refresh data control reloads saved changes. Records linked to order history are retained when deletion would break historical references.

5.3 Reports and Data Visualizations

The overview presents catalogue and order counts, delivered revenue, low-stock items, monthly revenue, order-status distribution, and recent activity. Additional report views summarize seller sales, pending returns, payment results, inventory, and customer and order activity. The charts and KPI cards use the included synthetic records.

5.4 Dashboard Screenshots

The two screenshots below document representative application workflows. The overview demonstrates the operational dashboard; the checkout view shows the applied coupon, discount, shipping, and updated total before an order is placed.

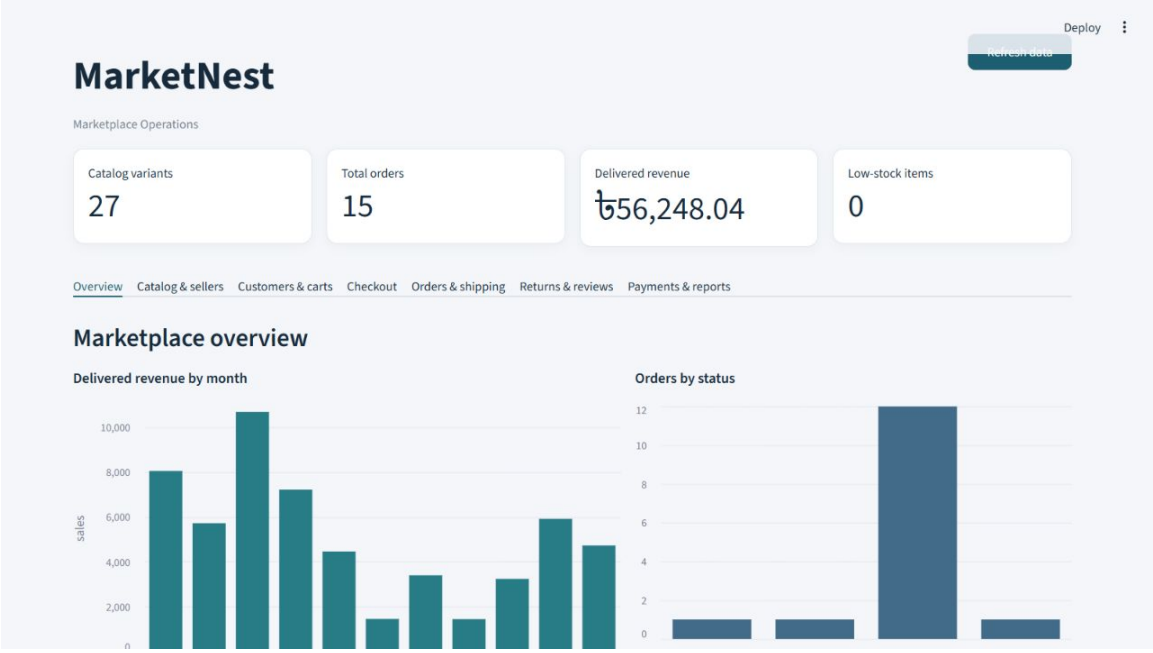


Figure 5.1: MarketNest overview with KPI cards, monthly sales, and order-status charts.

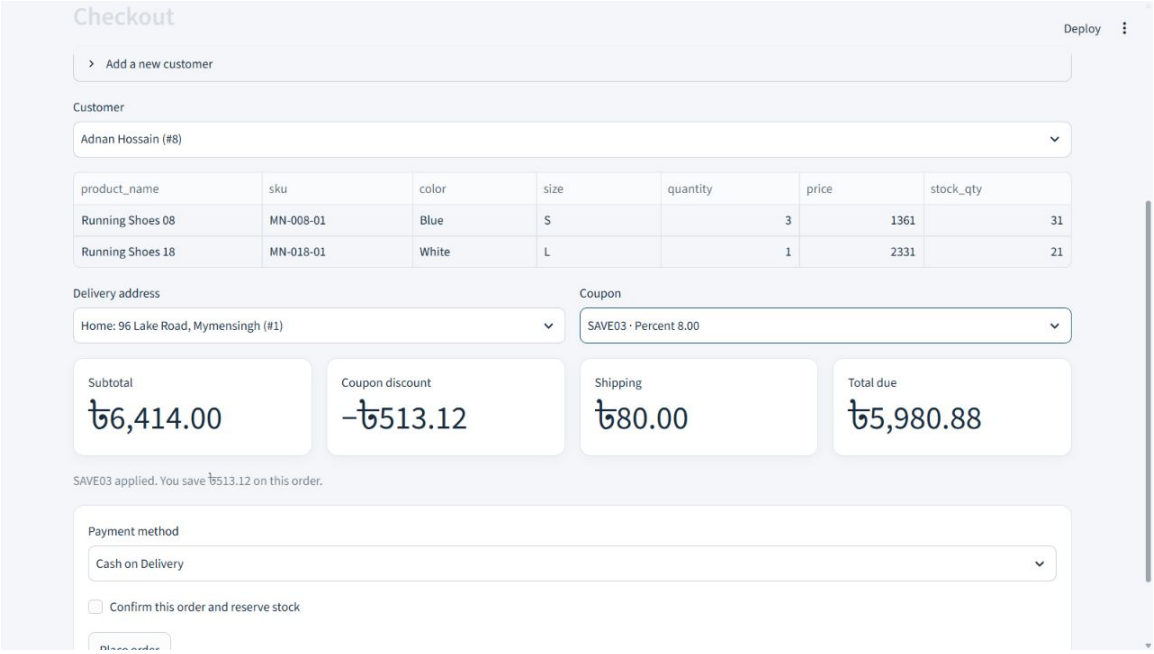


Figure 5.2: Checkout preview showing coupon savings and the updated total before order placement.

5.5 System Demonstration

For a classroom demonstration, the prepared synthetic database can be shown in this sequence:

1. Open the overview and explain the KPI cards, sales chart, and order-status summary.
2. Open the catalogue and create or update a seller, product, and size/color variant.
3. Search and filter catalogue records, update inventory, and demonstrate guarded deletion.
4. Select a customer and product variants at checkout; apply a coupon and explain the live subtotal, discount, shipping, and total.
5. Place an order and show its pending state, then advance it through confirmation and shipment.
6. Open returns and reviews to show the delivery-window rules and refund workflow.
7. Open payment and payout reports and explain the seller commission calculation.
8. Finish by showing the SQL views, constraints, and test results in the project files.

The application runs locally after the SQL scripts are installed, dependencies are installed, and a local `.env` is configured. The project root README contains the exact launch command and setup steps. The application is an academic operations demonstration; it is not a production marketplace, has no production authentication, and does not connect to a live payment gateway or courier API.

Chapter 6

Testing

6.1 Test Strategy and Results

The project documents 34 test cases and includes a Python runner that executes them against a live MySQL instance. Coverage includes valid input, invalid input, duplicate data, foreign-key violations, boundary values, transaction rollback, unauthorized access, triggers, stored procedures, views, and query output. The included observed-results file records 34 PASS and 0 FAIL for the configured MarketNest database on 10 November 2026. The test runner can be run again on another installation; results depend on the installed schema and local database configuration.

Table 6.1: Observed test execution summary

Result	Count	Status
Passed	34	PASS
Failed	0	PASS
Total	**34**	**PASS**

6.2 Detailed Test Cases

Each test case below includes the input or action, expected outcome, observed result, and status specified by the course brief. The detailed CSV files in `tests/` are included in the project archive and can be reviewed independently.

Table 6.2: Test case inputs, expected results, observed results, and status

Test ID	Input / action	Expected, observed result and status
TC01	Count all base tables and rows in the seeded database.	**Expected:** 18 tables; minimum count >= 10 **Observed:** 18 relations checked; minimum row count is 10 **Status:** PASS
TC02	Set stock_qty to -1 for product 1, variant 1.	**Expected:** CHECK constraint rejects update **Observed:** Database rejected invalid input (error 3819) **Status:** PASS

Test ID	Input / action	Expected, observed result and status
TC03	Insert a customer using the existing customer01@example.test email.	**Expected:** UNIQUE constraint rejects insert **Observed:** Database rejected invalid input (error 1062) **Status:** PASS
TC04	Insert a cart item with nonexistent cart_id 999999.	**Expected:** Foreign key rejects insert **Observed:** Database rejected invalid input (error 1452) **Status:** PASS
TC05	Insert a product with nonexistent seller_id 999999.	**Expected:** Foreign key rejects insert **Observed:** Database rejected invalid input (error 1452) **Status:** PASS
TC06	Insert an order for customer 1 with missing address_no 999.	**Expected:** Composite foreign key rejects insert **Observed:** Database rejected invalid input (error 1452) **Status:** PASS
TC07	Insert a product variant using the existing SKU MN-001-01.	**Expected:** UNIQUE constraint rejects insert **Observed:** Database rejected invalid input (error 1062) **Status:** PASS
TC08	Insert a Percent coupon with discount_value 150.	**Expected:** CHECK constraint rejects insert **Observed:** Database rejected invalid input (error 3819) **Status:** PASS
TC09	Insert an order with subtotal 100, shipping 50, and inconsistent total 120.	**Expected:** CHECK constraint rejects insert **Observed:** Database rejected invalid input (error 3819) **Status:** PASS
TC10	Insert a review with rating 6 (allowed range is 1 to 5).	**Expected:** CHECK constraint rejects insert **Observed:** Database rejected invalid input (error 3819) **Status:** PASS
TC11	Insert a review for a non-delivered order item (order 15, line 1).	**Expected:** Trigger rejects insert **Observed:** Database rejected invalid input (error 1644) **Status:** PASS
TC12	Insert a shipment using the existing tracking number TRK-0001.	**Expected:** UNIQUE constraint rejects insert **Observed:** Database rejected invalid input (error 1062) **Status:** PASS
TC13	Reserve one in-stock variant, check stock decreased by one, then roll back.	**Expected:** Procedure inserts line and decreases stock **Observed:** Stock decreased by one for an inserted order line; transaction rolled back **Status:** PASS
TC14	Call the reserve procedure with invalid product/variant and quantity 999999.	**Expected:** Procedure rejects oversell **Observed:** Database rejected invalid input (error 1644) **Status:** PASS
TC15	Check information_schema for the three required reporting views.	**Expected:** All 3 views listed in information_schema **Observed:** All three reporting views are present **Status:** PASS
TC16	Check information_schema for three procedures and at least 13 triggers.	**Expected:** 3 procedures and 13 triggers installed **Observed:** Found 3 required procedures and 13 triggers **Status:** PASS

Test ID	Input / action	Expected, observed result and status
TC17	Call seller payout for seller 1 and 2025 dates; roll back the test.	**Expected:** Returns a payout ID; test transaction rolls back **Observed:** Payout procedure created a result inside a rolled-back test transaction **Status:** PASS
TC18	Change stock, cause a CHECK failure, and verify rollback restores stock.	**Expected:** Stock value equals original after rollback **Observed:** Constraint failure was rolled back; original stock remained unchanged **Status:** PASS
TC19	Insert a payment with the existing transaction reference PAY-MN-0001.	**Expected:** UNIQUE constraint rejects insert **Observed:** Database rejected invalid input (error 1062) **Status:** PASS
TC20	Connect using a deliberately invalid MySQL username and password.	**Expected:** MySQL denies the deliberately invalid login **Observed:** Database rejected the deliberately invalid account (error 1045) **Status:** PASS
TC21	Request one more return unit than purchased for order 12, line 1.	**Expected:** Trigger rejects excess quantity **Observed:** Database rejected invalid input (error 1644) **Status:** PASS
TC22	Request a return eight days after delivery for order 12.	**Expected:** Trigger rejects an expired request **Observed:** Database rejected invalid input (error 1644) **Status:** PASS
TC23	Insert a cart item with quantity zero.	**Expected:** CHECK constraint rejects quantity zero **Observed:** Database rejected invalid input (error 3819) **Status:** PASS
TC24	Query the catalogue view for variants with available stock.	**Expected:** At least one available variant is returned **Observed:** Catalog query returned 33 available variants **Status:** PASS
TC25	Update seller 1 payout dates into an overlapping period.	**Expected:** Trigger rejects overlap with another seller payout **Observed:** Update trigger rejected overlapping payout dates (error 1644) **Status:** PASS
TC26	Update return request 1 quantity to 4, above its purchased quantity.	**Expected:** Trigger rejects excessive return units **Observed:** Database rejected invalid input (error 1644) **Status:** PASS
TC27	Insert an order with coupon 1 but an incorrect zero discount.	**Expected:** Trigger rejects an incorrect coupon discount **Observed:** Database rejected invalid input (error 1644) **Status:** PASS
TC28	Insert a category whose parent_category_id is its own ID.	**Expected:** Trigger rejects a category whose parent is itself **Observed:** Database rejected invalid input (error 1644) **Status:** PASS

Test ID	Input / action	Expected, observed result and status
TC29	Reserve stock and cancel an unshipped order; verify order, payment, and stock, then clean up.	**Expected:** Procedure cancels an unshipped order, marks payment failed, and restores inventory **Observed:** Cancellation procedure updated order/payment and restored reserved stock **Status:** PASS
TC30	Change order 1 from Delivered to Pending.	**Expected:** Trigger blocks a Delivered-to-Pending transition **Observed:** Database rejected invalid input (error 1644) **Status:** PASS
TC31	Insert a seller 1 payout for June 2026 overlapping an existing payout.	**Expected:** Insert trigger rejects an overlapping payout date range **Observed:** Database rejected invalid input (error 1644) **Status:** PASS
TC32	Update category 1 to use itself as its parent.	**Expected:** Update trigger rejects self-parenting **Observed:** Database rejected invalid input (error 1644) **Status:** PASS
TC33	Refund approved return 4; verify inventory restoration, then roll back.	**Expected:** Refund transition trigger restores returned inventory **Observed:** Refund transition restored returned units to inventory; transaction rolled back **Status:** PASS
TC34	Change rejected return 7 to Approved.	**Expected:** Trigger rejects Rejected-to-Approved transition **Observed:** Database rejected invalid input (error 1644) **Status:** PASS

6.3 Test Coverage and Error Handling

The tests verify that valid stock reservations decrease inventory, a valid cancellation restores it, and rollback preserves the original data after failure. Negative stock, zero quantity, duplicate identifiers, invalid references, invalid totals, out-of-range ratings, invalid coupon discounts, self-parenting categories, invalid status transitions, excess return quantities, and out-of-window returns are rejected by database constraints or triggers. A deliberately invalid database account is refused by MySQL. The application displays database validation errors to the operator and rolls back incomplete transactional operations.

Chapter 7

Conclusion

7.1 Achievements

MarketNest implements a relational database for a multi-vendor e-commerce marketplace using MySQL and a Python/Streamlit operations dashboard. The project defines 18 meaningful relations, includes synthetic sample records, enforces primary and foreign keys and integrity constraints, and provides 29 business queries, three views, three stored procedures, and thirteen triggers. Transaction handling protects checkout and cancellation workflows. The dashboard demonstrates catalogue, customer, cart, order, shipping, return, review, payment, payout, reporting, and chart operations.

The normalization example documents UNF-to-1NF, 2NF, and 3NF decomposition, with a BCNF assessment under stated functional dependencies. The recorded database test run completed with 34 passing cases. Together, the schema, SQL features, test evidence, and application demonstrate database design, implementation, transaction processing, and integration.

7.2 Limitations

- No live external payment gateway is integrated; payment outcomes are stored locally.
- No live courier booking or carrier-charge calculation is implemented.
- Dashboard users are not authenticated through a production login system.
- The demonstration uses synthetic data and a locally configured database account.
- Multi-warehouse inventory and a complete bank settlement/refund ledger are outside the current scope.

7.3 Future Improvements

Possible extensions include role-based application login, seller-specific access controls, a payment gateway sandbox, courier API integration, stock reservation expiry, audit logs, automated backups, multi-warehouse inventory, and a complete accounting and settlement ledger.

References

[1] ECE 2216, *Real-World Database System Design and Implementation*, course project brief, 2026.

[2] Mustakim Al Siyam, *MarketNest: Design of a Multi-Vendor E-Commerce Database System*, ECE 2215 assignment report, Rajshahi University of Engineering & Technology, 2026.

[3] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.

[4] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill, 2020.

[5] Oracle, *MySQL 8.0 Reference Manual*, https://dev.mysql.com/doc/refman/8.0/en/.

[6] Streamlit, *Streamlit Documentation*, https://docs.streamlit.io/.